МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №4

"Точність формул для чисельного диференціювання. Метод Рунге-Ромберга. Метод Ейткена"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

ас. Каськун О. Д.

Львів-2026

Мета роботи: Дослідити точність чисельного диференціювання та перевірити уточнення похідної методами Рунге-Ромберга й Ейткена.

Теоретичні відомості:

Центральна різниця $D(h) = [M(x0+h) - M(x0-h)]/(2h)$ має похибку $O(h^2)$. Зменшення h знижує похибку апроксимації, але посилює вплив округлення. Уточнення $D_R = D(h) + [D(h)-D(2h)]/3$ усуває головний член похибки; для симетричної формули залишкова похибка має порядок $O(h^4)$. Метод Ейткена використовує три наближення. Порядок оцінюється як $p = \log2(|[D(4h)-D(2h)]/[D(2h)-D(h)]|)$.

Виконання роботи

Репозиторій на GitHub:
https://github.com/RakhimAbdulkhanov/numerical_methods_2026

1. Задав $M(t) = 50\exp(-0.1t) + 5\sin(t)$. Аналітично отримав $M'(t) = -5\exp(-0.1t) + 5\cos(t)$. Взяв x0 = 1, як у розібраному прикладі завдання. У самому переліку кроків окремого значення x0 немає.

```python
import math

import matplotlib.pyplot as plt
import numpy as np


X0 = 1.0
H = 1e-3
```

```python
def moisture(t):
    return 50 * math.exp(-0.1 * t) + 5 * math.sin(t)

def derivative(t):
    return -5 * math.exp(-0.1 * t) + 5 * math.cos(t)

def central_difference(function, x, h):
    if not math.isfinite(h) or h <= 0:
        raise ValueError("крок має бути додатним і скінченним")
    return (function(x + h) - function(x - h)) / (2 * h)

def runge_romberg(fine, coarse, order=2):
    return fine + (fine - coarse) / (2**order - 1)

def aitken(fine, medium, coarse):
    first = medium - fine
    second = coarse - medium
    denominator = second - first
    if denominator == 0 or first == 0 or second == 0:
        raise ValueError("недостатньо різниці між наближеннями")
    # різницева форма не віднімає квадрати близьких чисел
    refined = fine - first * first / denominator
    order = math.log2(abs(second / first))
    return refined, order
```

2. Перевірив кроки від 1e-20 до 1e3 з мантисами 1..9. Це скінченний пошук, тому h0 означає найкращий перевірений крок, а не точний глобальний мінімум. Для порівняння методів використав заданий h = 1e-3 та кроки 2h і 4h.

```python
def experiment():
    exact = derivative(X0)
    # перевіряєм мантиси 1..9 для кожного десяткового порядку
    steps = np.array(sorted({m * 10.0**e
                             for e in range(-20, 4)
                             for m in range(1, 10)
                             if m * 10.0**e <= 1e3}))
    values = np.array([central_difference(moisture, X0, h)
                       for h in steps])
    errors = np.abs(values - exact)
    best = int(np.argmin(errors))
    fine, medium, coarse = [central_difference(moisture, X0, k * H)
                            for k in (1, 2, 4)]
    rr = runge_romberg(fine, medium)
    improved, order = aitken(fine, medium, coarse)
    comparisons = [
        ("D(h)", fine), ("D(2h)", medium), ("D(4h)", coarse),
        ("Runge-Romberg", rr), ("Aitken", improved),
    ]
    return dict(exact=exact, steps=steps, errors=errors,
                best_h=float(steps[best]), best_error=float(errors[best]),
                comparisons=comparisons, order=order)
```

Параметр	Значення
M′(1)	-1.822675560839099
h0	5e-06
R0	1.359519e-10
Оцінка p	2.000000933

3. Порівняв наближення та їхні абсолютні похибки з аналітичною похідною. Значення рахував без попереднього округлення. Різницевий запис формули Ейткена алгебраїчно рівнозначний формулі з методички та не потребує віднімання квадратів близьких чисел.

```python
def print_results(result):
    print(f"x0 = {X0:g}; h = {H:g}")
    print(f"M'(1) = {result['exact']:.15f}")
    print(f"h0 = {result['best_h']:.0e}")
    print(f"R0 = {result['best_error']:.6e}")
    print(f"{'method':<15} {'derivative':>20} {'absolute error':>15}")
    for name, value in result['comparisons']:
        error = abs(value - result['exact'])
        print(f"{name:<15} {value:20.15f} {error:15.6e}")
    print(f"p = {result['order']:.9f}")
```

```
PS C:\Users\ponti> python numerical_methods_2026/lab4/main.py
x0 = 1; h = 0.001
M'(1) = -1.822675560839099
h0 = 5e-06
R0 = 1.359519e-10
method                    derivative  absolute error
D(h)              -1.822676018633729    4.577946e-07
D(2h)             -1.822677392008032    1.831169e-06
D(4h)             -1.822682885508797    7.324670e-06
Runge-Romberg     -1.822675560842294    3.195888e-12
Aitken            -1.822675560842689    3.590461e-12
p = 2.000000933

PS C:\Users\ponti> _
```

Отримав R1 = 4.577946e-07, R2 = 3.195888e-12, R3 = 3.590461e-12. Обидва уточнення зменшили похибку порівняно з D(h). Ейткен не виявився точнішим за Рунге-Ромберга на цих кроках. Оцінка p близька до 2, як і очікується для центральної різниці.

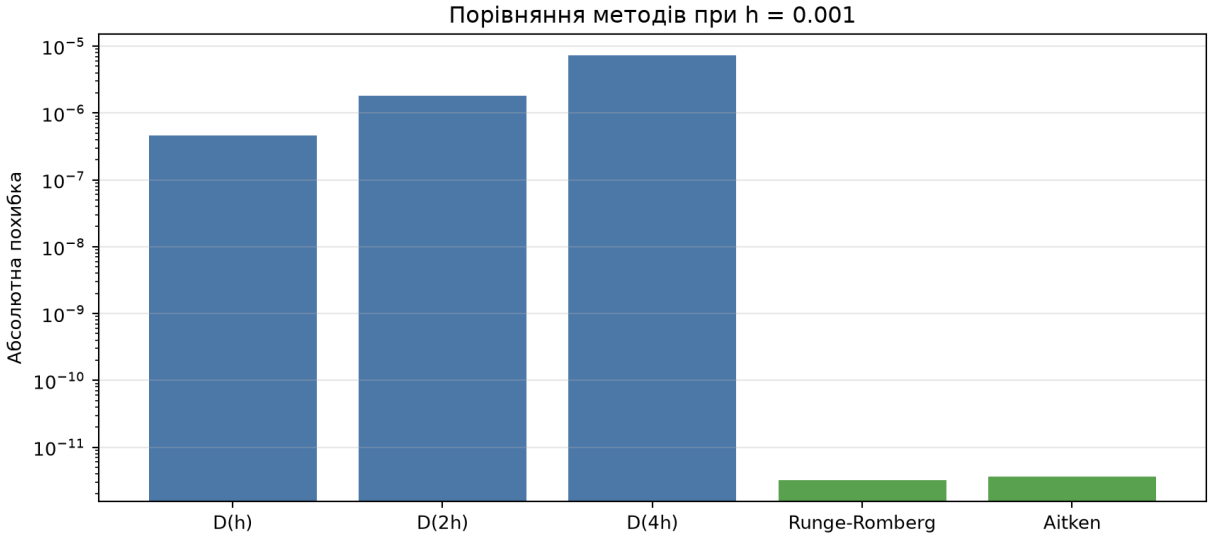
4. Побудував залежність похибки від h. На великих кроках домінує апроксимаційна похибка, на малих - віднімання близьких значень та округлення. Ділянку h > 1 використав лише як чисельний експеримент: вона виходить за фізичну область t >= 0.

```python
def make_plots(result):
    fig, ax = plt.subplots(figsize=(9, 5), layout="constrained")
    ax.loglog(result['steps'], result['errors'], ".-", markersize=3)
    ax.scatter(result['best_h'], result['best_error'], color="red",
               zorder=5, label=f"найкращий перевірений h = {result['best_h']:.0e}")
    ax.set(xlabel="Крок h", ylabel="Абсолютна похибка",
           title="Центральна різниця: апроксимація та округлення")
    ax.grid(True, which="both", alpha=0.3)
    ax.legend()
```



5. Побудував порівняння похибок для заданого базового кроку. Логарифмічна вісь дає змогу побачити різницю між прямим наближенням і уточненнями.

```
    fig2, ax2 = plt.subplots(figsize=(9, 4), layout="constrained")
    labels = [name for name, _ in result['comparisons']]
    errors = [abs(value - result['exact'])
              for _, value in result['comparisons']]
    ax2.bar(labels, errors, color=["#4c78a8"] * 3 + ["#59a14f"] * 2)
    ax2.set(yscale="log", ylabel="Абсолютна похибка",
            title="Порівняння методів при h = 0.001")
    ax2.grid(True, axis="y", alpha=0.3)
    return [fig, fig2]
```



Перевірка реалізації. Запустив автоматичні тести: правильність базових формул, перевірку некоректних параметрів та узгодженість результатів експерименту.

```
def main():
    result = experiment()
    print_results(result)
    make_plots(result)
    plt.show()
```

```
import math
import unittest

import main


class DifferentiationTests(unittest.TestCase):
    def test_central_difference_polynomial(self):
        self.assertAlmostEqual(
            main.central_difference(lambda x: x*x, 3.0, 0.25), 6.0)

    def test_runge_cancels_quadratic_error(self):
        self.assertAlmostEqual(main.runge_romberg(2.01, 2.04), 2.0)

    def test_aitken_geometric_error(self):
        value, order = main.aitken(2.01, 2.04, 2.16)
        self.assertAlmostEqual(value, 2.0)
        self.assertAlmostEqual(order, 2.0)

    def test_invalid_and_degenerate_input(self):
        for step in (0, -1, math.inf, math.nan):
            with self.assertRaises(ValueError):
                main.central_difference(main.moisture, 1, step)
        with self.assertRaises(ValueError):
            main.aitken(1, 1, 1)

    def test_experiment(self):
        result = main.experiment()
        self.assertEqual(result['steps'][0], 1e-20)
        self.assertEqual(result['steps'][-1], 1e3)
        self.assertEqual(result['best_error'], min(result['errors']))
        self.assertAlmostEqual(result['order'], 2.0, places=4)
```

```
        errors = [abs(value - result['exact'])
                  for _, value in result['comparisons']]
        self.assertLess(errors[3], errors[0])
        self.assertLess(errors[4], errors[0])


if __name__ == '__main__':
    unittest.main()
```

```
PS C:\Users\ponti> python -m unittest discover -s numerical_methods_2026/lab4 -p test_main.py -v
test_aitken_geometric_error (test_main.DifferentiationTests.test_aitken_geometric_error) ... ok
test_central_difference_polynomial (test_main.DifferentiationTests.test_central_difference_polynomial) ...
ok
test_experiment (test_main.DifferentiationTests.test_experiment) ... ok
test_invalid_and_degenerate_input (test_main.DifferentiationTests.test_invalid_and_degenerate_input) ... ok
test_runge_cancels_quadratic_error (test_main.DifferentiationTests.test_runge_cancels_quadratic_error) ...
ok

----------------------------------------------------------------------
Ran 5 tests in 0.002s

OK
PS C:\Users\ponti> _
```

Висновок:

Дослідив чисельне диференціювання функції вологості. Для перевіреної сітки найменшу похибку 1.360e-10 отримав при h0 = 5e-06. При h = 1e-3 методи Рунге-Ромберга та Ейткена зменшили похибку до 3.196e-12 і 3.590e-12 відповідно. Оцінка p = 2.000001 підтвердила другий порядок центральної різниці. Переконався, що надмірне зменшення кроку не гарантує кращого результату.